

PRODUCT REQUIREMENT DOCUMENT (PRD)

Project: Core Banking Customer 360 & Hyper-Personalization Engine

Document Version: 1.0

Status: Approved for Implementation

Owner: Data & Analytics Product Team

Tech Stack: Google Cloud Storage (GCS), Cloud Composer 2 (Apache Airflow 2.x), Google Cloud Dataform, BigQuery, BigQuery Policy Tags, IAM

1. Introduction & Objectives

1.1 Purpose

This Product Requirement Document (PRD) outlines the detailed technical specs, user requirements, and system behaviors for building the Core Banking Customer 360 & Hyper-Personalization Engine. This system acts as a bridges between legacy AS400 core banking transactional systems and modern marketing/analytical engines running on Google Cloud Platform (GCP).

1.2 System Scope

The scope of this product includes:

- **Ingestion Automation:** Monitoring and loading raw AS400 text-based files from GCS into BigQuery Bronze datasets.
- **Medallion Pipelines:** Orchestrating SQL-based transformations inside GCP Dataform to structure Bronze raw tables, conformed Silver dimensional models, and Gold Customer 360 analytical marts.
- **Security & Governance Enforcement:** Declaratively applying Policy Tags to PII columns within Dataform compilation runs and configuring dynamic data masking in BigQuery.
- **Orchestration & Verification:** Using Cloud Composer to coordinate end-to-end execution, evaluate native Dataform assertions, and alert engineering teams of exceptions.

2. Target User Personas

To guide system boundaries and feature sets, we design for three distinct internal user personas:

2.1 Persona 1: The Data Engineer (DE)

- **Goal:** Build, maintain, and debug reliable data pipelines with minimal operational overhead.
- **Needs:** Clear lineage graphs, automated dependency resolution, schema evolution protections, and real-time failure alerts when data structures change.
- **Access Level:** Read/Write on all BigQuery datasets and full admin on Cloud Composer/Dataform.

2.2 Persona 2: The Data Analyst / BI Developer (DA)

- **Goal:** Create reports, segment clients, and query demographic parameters without dealing with legacy column jargon.
- **Needs:** Highly denormalized tables, clean naming conventions (e.g., customer_id instead of CMCIF), clear metric definitions, and accurate data freshness indicators.
- **Access Level:** Standard query access to Gold layers. High PII fields must resolve in a masked or anonymized format.

2.3 Persona 3: The Campaign Manager / CRM Marketer (CM)

- **Goal:** Build target audiences and sync Next-Best-Action metrics to email, SMS, and banking portal campaigns.
- **Needs:** Clear customer segments, propensity scores, and accessible customer contact information (for target users).
- **Access Level:** Accesses final audiences through visualization platforms or downstream CRM tooling. Requires highly governed access to unmasked customer information.

3. Product Features & User Stories (Epics)

CORE PRODUCT FEATURES		
[EPIC 1: Ingestion] → [EPIC 2: Transformations] → [EPIC 3: Governance]		
- GCS to Bronze	- Dataform Dependency	- Policy Tag Taxon
- Sensor verification	- Silver Dim Modeling	- Dynamic Masking
- Append-only loads	- Gold 360 Aggregation	- IAM Restrict

Epic 1: GCS-to-Bronze Ingestion & Sensing

- **FR-1.1:** The orchestrator MUST automatically detect files delivered from the on-premise AS400 core database to Google Cloud Storage.
- **FR-1.2:** Ingestion tasks MUST operate incrementally or via full-overwrite depending on file

type, tracking ingestion lineage.

- *User Story:* As a Data Engineer, I want the system to wait for all four daily AS400 core files to arrive in GCS before kicking off the ingestion so that downstream transforms do not process incomplete files.

Epic 2: Dataform Transform Engine (Silver & Gold Layers)

- **FR-2.1:** Transformations **MUST** translate abbreviated legacy column names to consistent, lowercase snake_case standard formats.
- **FR-2.2:** Legacy number-based dates (Format: YYYYMMDD) **MUST** be safely cast to native BigQuery DATE types.
- **FR-2.3:** Downstream dependencies **MUST** automatically update if upstream tables modify.
- *User Story:* As a Data Analyst, I want the Customer 360 mart to combine savings, deposits, cards, and loan tables into a single wide view so that I do not have to write complex multi-table joins for every analysis.

Epic 3: Dynamic PII Protection (Column-Level Security)

- **FR-3.1:** Customer names, phone numbers, addresses, and credit card numbers **MUST** be tagged with centralized Policy Tags.
- **FR-3.2:** Unauthorized queries executing against protected datasets **MUST** automatically return masked, hashed, or NULL values depending on the specific policy.
- *User Story:* As a Campaign Manager, I want to see actual cleartext phone numbers of target customers so that I can dispatch personalized SMS messages, while standard business query accounts only see masked outputs.

Epic 4: Workflow Orchestration & Data Quality Control

- **FR-4.1:** Dataform SQLX files **MUST** implement native assertions ensuring primary key uniqueness and non-null rules.
- **FR-4.2:** Composer **MUST** catch assertion failures and instantly pause execution before writing invalid data to production Gold marts.
- *User Story:* As a Data Engineer, I want the workflow run to fail if a duplicate customer_id is found so that reporting dashboards do not count transactions multiple times.

4. Technical Specifications & Data Flow

4.1 Cloud Infrastructure Directory Mapping

```
gcp-analytics-project/  
├── Storage Buckets  
│   └── gs://as400-core-data-drop/ (Daily files: customer_mast, account_mast, cc_txn,  
loan_mast)  
├── BigQuery Datasets  
│   ├── demo_bronze_as400/ (Managed via Composer file loads; raw tables)  
│   └── demo_silver_banking/ (Managed via Dataform SQLX models; conformed tables)
```

```

|   └─ demo_gold_analytics/ (Managed via Dataform SQLX models; analytical tables)
└─ Cloud Composer Environment
    └─ gs://us-central1-composer-bucket/dags/daily_as400_medallion_load.py

```

4.2 Detailed Transformation & Metric Derivation Formulas

To calculate analytical segments inside mart_customer_360.sqlx, the system must process conformed financial metrics using standard mathematical logic.

Debt-to-Deposit Ratio (DDR)

Calculates client debt leverage to protect against over-leveraging risk:

$$DDR = \frac{\sum \text{total_loan_outstanding}}{\max(0.01, \sum \text{total_savings_balance})}$$

Investment Propensity Score (IPS)

Calculates an initial 0-100 score indicating likelihood to invest in term deposits:

$$IPS = (w_1 \times N_{\text{savings}}) + (w_2 \times (1 - N_{\text{debt}}))$$

Where:

- $w_1 = 0.7$ (Savings weight), $w_2 = 0.3$ (Debt reduction weight)
- $N_{\text{savings}} = \min\left(1.0, \frac{\text{total_savings_balance}}{100,000}\right)$
- $N_{\text{debt}} = \min\left(1.0, \frac{\text{total_loan_outstanding}}{500,000}\right)$

5. Security & Governance Implementation Specification

5.1 Policy Tag Hierarchy & Taxonomy Config

A centralized taxonomy named Banking_Customer_Data_Classification will be deployed inside BigQuery Policy Tag Manager.

TAXONOMY: Banking_Customer_Data_Classification

- Highly_Sensitive_PII
 - Tag Name: PII_Name (Resource ID: taxonomy/812345/policyTags/98765)
 - Tag Name: PII_Phone (Resource ID: taxonomy/812345/policyTags/22334)
 - Tag Name: PII_Address (Resource ID: taxonomy/812345/policyTags/11223)
- Sensitive_Financials
 - Tag Name: Card_PAN (Resource ID: taxonomy/812345/policyTags/44556)

5.2 Dynamic Data Masking Configurations

The following policies must be attached to the respective policy tags inside BigQuery:

- 1. PII_Name Masking Policy (SHA-256 Hashing)
 - Query return format for unauthorized users:
 - Original: "John Smith" → Return: "a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3"
- 2. PII_Phone Masking Policy (Custom RegEx Redaction)
 - Query return format:
 - Original: "+1-555-0199" → Return: "XXXX-XXXX-0199"
- 3. PII_Address Masking Policy (Nullification)
 - Query return format:
 - Original: "123 Financial Way, Suite A" → Return: NULL
- 4. Card_PAN Masking Policy (First 12 Digit Redaction)
 - Query return format:
 - Original: "4111222233334444" → Return: "XXXXXXXXXXXX4444"

5.3 IAM Role Mapping Configuration

Access control to raw elements must be applied via Google Cloud IAM bindings:

Identity Group	Required BigQuery Permissions	Policy Tag Access Level	Purpose
gcp-data-engineers@bank.com	BigQuery Admin, Metadata Viewer	None (Masked in default queries)	Code deployment and system monitoring.

gcp-data-analysts@bank.com	BigQuery Job User, Data Viewer	Default Masking Policies	Creating analytical models and dashboard queries.
gcp-marketing-users@bank.com	BigQuery Job User, Data Viewer	Fine-Grained Reader on PII_Name, PII_Phone	Direct target list extraction for approved campaigns.
gcp-compliance-auditors@bank.com	BigQuery Data Viewer	Fine-Grained Reader on all tags	Performing verification and standard audits.

6. Functional Schemas & Dataform Configuration Specs

6.1 Silver Layer Schema (Example: dim_customers.sqlx)

To register the conformed customer dimension while ensuring automated policy tag bindings, write the following .sqlx parameters.

```
config {
  type: "table",
  schema: "demo_silver_banking",
  description: "Standardized customer records containing PII masking classifications",
  assertions: {
    uniqueKey: ["customer_id"],
    nonNull: ["customer_id"]
  },
  columns: {
    customer_id: "Unique CIF key.",
    full_name: {
      description: "Customer Name with strict hashing policies",
      policyTags:
["projects/gcp-project-id/locations/global/taxonomies/812345/policyTags/98765"]
    },
    address: {
      description: "Residential address with default nullification",
      policyTags: ["projects/gcp-project-id/locations/global/taxonomies/812345/policyTags/11223"]
    },
    phone_number: {
      description: "Primary telephone field",
```

```

    policyTags:
["projects/gcp-project-id/locations/global/taxonomies/812345/policyTags/22334"]
  }
}
}

```

```

SELECT
  TRIM(CMCIF) AS customer_id,
  TRIM(CMNAME) AS full_name,
  TRIM(CMADR1) AS address,
  PARSE_DATE('%Y%m%d', CAST(CMDOB AS STRING)) AS birth_date,
  UPPER(TRIM(CMSEG)) AS customer_segment,
  TRIM(CMPHNE) AS phone_number,
  CURRENT_TIMESTAMP() AS _ingested_at,
  _source_file
FROM
  ${ref("AS400_CUST_MAST")}

```

6.2 Gold Layer Schema (Example: mart_customer_360.sqlx)

This denormalized model leverages downstream references, executing after Silver objects build.

```

config {
  type: "table",
  schema: "demo_gold_analytics",
  description: "Unified analytical Customer 360 profiling table",
  assertions: {
    uniqueKey: ["customer_id"],
    nonNull: ["customer_id", "total_savings_balance"]
  }
}

```

```

WITH agg_savings AS (
  SELECT
    ac_cif AS customer_id,
    SUM(CASE WHEN account_type = 'SV' THEN balance ELSE 0 END) AS total_savings_balance,
    SUM(CASE WHEN account_type = 'DP' THEN balance ELSE 0 END) AS total_deposit_balance
  FROM
    ${ref("dim_accounts")}
  WHERE status = 'A'
  GROUP BY 1
),

```

```

agg_cards AS (
  SELECT
    customer_id,
    SUM(CASE WHEN transaction_type = 'DEBIT' AND transaction_date >=
DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY) THEN transaction_amount ELSE 0 END) AS
cc_spend_last_30_days,
    APPROX_TOP_COUNT(spending_category, 1)[OFFSET(0)].value AS top_spending_category
  FROM
    ${ref("fct_credit_card_transactions")}
  GROUP BY 1
),

```

```

agg_loans AS (
  SELECT
    customer_id,
    MAX(CASE WHEN loan_type = 'MORTGAGE' THEN TRUE ELSE FALSE END) AS
has_active_mortgage,
    SUM(outstanding_balance) AS total_loan_outstanding
  FROM
    ${ref("dim_loans")}
  GROUP BY 1
)

```

```

SELECT
  c.customer_id,
  c.full_name, -- Automatically inherits policy tags from dim_customers
  c.phone_number, -- Automatically inherits policy tags from dim_customers
  DATE_DIFF(CURRENT_DATE(), c.birth_date, YEAR) AS age,
  c.customer_segment,
  COALESCE(s.total_savings_balance, 0) AS total_savings_balance,
  COALESCE(s.total_deposit_balance, 0) AS total_deposit_balance,
  COALESCE(cc.cc_spend_last_30_days, 0) AS cc_spend_last_30_days,
  COALESCE(cc.top_spending_category, 'NONE') AS top_spending_category,
  COALESCE(l.has_active_mortgage, FALSE) AS has_active_mortgage,
  COALESCE(l.total_loan_outstanding, 0) AS total_loan_outstanding,

  -- Mathematical Indicators
  SAFE_DIVIDE(COALESCE(l.total_loan_outstanding, 0), COALESCE(s.total_savings_balance, 0))
AS debt_to_deposit_ratio,

  -- Target Segment Derivation
  CASE

```



```

    WHEN s.total_savings_balance > 250000 AND COALESCE(l.total_loan_outstanding, 0) = 0
    THEN 'HNW_INVESTOR'
    WHEN l.total_loan_outstanding > 200000 AND s.total_savings_balance < 10000 THEN
    'LEVERAGED_BORROWER'
    WHEN cc.cc_spend_last_30_days > 5000 THEN 'DIGITAL_SHOPPER'
    ELSE 'STANDARD_RETAIL'
    END AS propensity_score_segment

FROM
    ${ref("dim_customers")} c
    LEFT JOIN agg_savings s ON c.customer_id = s.customer_id
    LEFT JOIN agg_cards cc ON c.customer_id = cc.customer_id
    LEFT JOIN agg_loans l ON c.customer_id = l.customer_id

```

7. Non-Functional Requirements (NFRs)

- NFR-1: Freshness SLA**
 The downstream mart_customer_360 table must be updated and verified by 5:00 AM UTC every day.
- NFR-2: Execution Isolation**
 Dataform compiles must run on independent execution branches, ensuring that development work does not affect production schedules.
- NFR-3: Cost Efficiency**
 All SQLX transformations must use partitioning on ingestion timestamp (_ingested_at) or transaction dates (transaction_date) to prevent expensive full-table scans.
- NFR-4: Auditability**
 Every query executing against conformed datasets must be logged in GCP Cloud Logging (Audit Logs) to trace access to sensitive PII fields.

8. Testing & Validation Plan

Test Case ID	Scenario Description	Expected Output	Verification Method
TC-1.1	Raw text file has missing columns during GCS to Bronze run.	Pipeline execution fails during GCS load task, alerting Slack.	Trigger load with corrupt sample mock file.
TC-2.1	Run standard user query with	Name matches hash pattern; phone	Execute test query from BigQuery

	gcp-data-analysts@bank.com credentials.	is redacted; address returns NULL.	Workspace under mock user profile.
TC-2.2	Run campaign extraction query with gcp-marketing-users@bank.com credentials.	Cleartext names and phones are displayed correctly.	Execute test query under mock marketing profile.
TC-3.1	Account balances are updated with negative savings parameters.	Dataform custom assertion (assert_balanced_accounts) triggers failure, pausing pipeline execution.	Run pipeline run with synthetic corrupt values.

9. Implementation Roadmap

Phase 1: Foundation (Weeks 1-2)

- └─ Setup GCS Buckets & Ingestion Directories
- └─ Create Policy Tag Taxonomy & Define Masking Rules

Phase 2: Modeling (Weeks 3-4)

- └─ Initialize Dataform Project & Define Sources
- └─ Write dim_customers, dim_accounts, & mart_customer_360 SQLX models

Phase 3: Integration (Week 5)

- └─ Code Composer Airflow DAG with Dataform Operators
- └─ Verify Assertions & Pipeline Exceptions

Phase 4: Launch (Week 6)

- └─ Run UAT Testing across Analyst & Marketing user profiles
- └─ Signoff & Transition to Production Schedule